

AegisCare HMS — Internal API & RPC Reference

Document Title: PostgreSQL Database RPC Functions & Supabase Edge Functions Reference

Document Version: 1.0.0

Runtime Environment: Supabase PostgreSQL 15+ & Deno Serverless Edge Runtime

Target Environment: Kenya MoH Level 1–6 Facilities

Part 1: Supabase PostgreSQL RPC Functions

accrue_daily_bed_charges

Purpose: Iterates over all active inpatient admissions across general, maternity, pediatric, and surgical wards, calculating and posting a daily bed charge line item to each patient's open invoice.

Parameters: None (**void**).

Returns: **void**

Called from: Nightly **pg_cron** daemon (`0 21 * UTC = 00:00 EAT`) and manual administrative batch trigger (`src/routes/admin.wards.tsx`).

Business logic:

- Queries **admissions** where **status** = 'admitted'.
 - Joins **wards** to extract the active **daily_rate** and **ward_name**.
 - Finds the primary open invoice linked to **admissions.encounter_id**.
 - Appends a line item to **invoice_line_items** with **item_type** = 'bed_day', **source_id** = **admission_id**, **quantity** = 1, and **amount** = **daily_rate**.
 - Description formatted as: "[Ward Name] - bed charge (YYYY-MM-DD)".
-

accrue_daily_icu_charges

Purpose: Accrues specialized daily intensive care monitoring and equipment staging surcharges for all patients currently occupying beds within the Intensive Care Unit (ICU) or High Dependency Unit (HDU).

Parameters: None (**void**).

Returns: **void**

Called from: Nightly **pg_cron** daemon.

Business logic:

- Queries active admissions specifically located in wards where **ward_type** = 'icu'.
 - Applies the intensive care unit daily tariff to the active encounter invoice.
 - Distinguishes specialized ICU monitoring charges from standard baseline bed charges.
-

accrue_daily_mortuary_charges

Purpose: Automatically calculates and posts daily cold-storage refrigeration fees for all deceased bodies currently held in the hospital mortuary.

Parameters: None (**void**).

Returns: **void**

Called from: Nightly **pg_cron** daemon (`5 21 * UTC = 00:05 EAT`) and Mortuary Module (`supabase/migrations/20260811000006_sprint_b3_mortuary_external_billing.sql`).

Business logic:

- Iterates over **mortuary_records** where **released_at** IS NULL.
- If the body originated internally (**body_source** = 'internal'), locates the linked hospital invoice.
- If the body was admitted externally (**body_source** = 'external'), locates or provisions the dedicated commercial mortuary invoice via **create_external_mortuary_invoice**.

- Inserts a line item into `invoice_line_items` with `item_type = 'mortuary_storage'` and standard daily storage fee (default KSh 1,000/day).

archive_old_audit_logs

Purpose: Archives audit log records older than two years by moving them to the long-term `audit_log_archive` table, satisfying Kenya's 20-year medical record retention mandate while optimizing primary transactional query performance.

Parameters: None (`void`).

Returns: `void`

Called from: Nightly `pg_cron` daemon (`0 23 * UTC = 02:00 EAT`)

(`supabase/migrations/20260806101050_e8638a1f-75b6-42c3-9a64-be6489877920.sql`).

Business logic:

- Calculates cutoff threshold: `v_cutoff := now() - INTERVAL '2 years'`.
- Deletes records where `changed_at < v_cutoff` from `public.audit_log` using a `RETURNING` clause.
- Inserts deleted rows atomically into `public.audit_log_archive`.
- Logs execution run metrics (rows moved, oldest timestamp, newest timestamp, execution status) into `audit_archive_runs`.

charge_icu_admission_fee

Purpose: Posts the mandatory, one-time specialized institutional ICU admission fee when an inpatient is admitted or transferred into an Intensive Care Unit ward.

Parameters:

- `p_encounter_id (uuid)` — The unique identifier of the active clinical encounter.
- `p_ward_id (uuid)` — The target ward identifier.

Returns: `void`

Called from: Inpatient Admission Dialog (`src/routes/rooms.$id.tsx` line 3997).

Business logic:

- Validates whether the target ward has `ward_type = 'icu'`.
- Locates the active invoice linked to `p_encounter_id`.
- Checks whether an `icu_admission_fee` line item has already been posted to avoid duplicate surcharges.
- Inserts a line item into `invoice_line_items` with `item_type = 'procedure'`, description '`ICU Admission & Specialized Staging Surcharge`', and standard institutional rate (e.g., KSh 5,000).

process_dialysis_session_billing

Purpose: Processes completed hemodialysis session clinical logs, records consumable inventory deductions, and posts itemized billing charges in a single atomic database transaction.

Parameters:

- `p_patient_id (uuid)` — The patient UUID.
- `p_room_id (uuid)` — The Dialysis Unit room UUID.
- `p_session_id (uuid)` — The completed `dialysis_sessions` record UUID.
- `p_items (jsonb)` — An array of consumable objects, each containing `item_id`, `name`, `quantity`, and `unit_price`.

Returns: `void`

Called from: Dialysis Nursing Workbench (`src/routes/rooms.$id.tsx` line 6597).

Business logic:

- Resolves the active encounter for the patient in the dialysis room.
- Iterates over the `p_items` JSON array:
- Invokes `record_stock_usage` to deduct each consumable from the Dialysis Store (`d38d45c7-20f2-4e1b-9279-8cb5bf567cd1`).
- Appends corresponding consumable line items to `invoice_line_items`.
- Appends the base Hemodialysis Session Procedure Fee line item.
- Recalculates invoice totals via database triggers.

auto_generate_sha_claim

Purpose: Automatically compiles a structured Social Health Authority (SHA) claim when an encounter is signed or closed for a patient registered under `payment_mode = 'sha_shif'`.

Parameters: Trigger function (executes on `UPDATE OF status ON encounters`).

Returns: `trigger` (or executed via RPC).

Called from: Database trigger `trg_auto_generate_sha_claim` on `encounters` (`supabase/migrations/20260813000003_sprint4_sha_claim_guards.sql`).

Business logic:

- Evaluates whether `NEW.payment_mode = 'sha_shif'` and `NEW.status = 'signed'`.
- Validates that at least one ICD-11 primary diagnosis exists in `encounter_diagnoses`.
- Classifies fund subtype (`phf`, `shif`, `eccif`) based on facility MoH level and clinical service room.
- Creates a record in `sha_claims` with status `'draft'`.
- Iterates over billable diagnostic and procedural items, mapping them to SHA tariff codes in `sha_claim_items`.
- Invokes `build_fhir_claim()` to compile the attached FHIR Claim resource JSON.

log_break_glass_access

Purpose: Executes an emergency security override, allowing authorized medical practitioners to access restricted or un-consented patient health records during acute emergencies while logging an immutable forensic audit event.

Parameters:

- `p_patient_id (uuid)` — The patient record identifier accessed.
- `p_justification (text)` — Mandatory written clinical justification for the override.
- `p_accessed_by (uuid)` — The authenticated practitioner UUID.
- `p_accessor_email (text)` — The practitioner's authenticated email address.

Returns: `void`

Called from: Clinical Consultation UI (`src/routes/rooms.$id.tsx`, `supabase/migrations/20260806000001_sprint8a_break_glass.sql`).

Business logic:

- Validates that `p_justification` is non-null and not empty; raises an exception if missing.
- Inserts an immutable security event into `public.audit_log` with `action = 'BREAK_GLASS'`, `table_name = 'patients'`, `record_id = p_patient_id`, and `new_data` capturing justification text, accessor email, and timestamp.
- Function defined as `SECURITY DEFINER` with execution granted to `authenticated`.

audit_trigger_fn

Purpose: Universal PostgreSQL database trigger function that captures table-level data mutations (INSERT, UPDATE, DELETE) and writes immutable before/after snapshots into `audit_log`.

Parameters: Trigger context (`TG_TABLE_NAME`, `TG_OP`, `NEW`, `OLD`).

Returns: `trigger`

Called from: Attached triggers across 15+ database tables (`encounters`, `patients`, `admissions`, `prescriptions`, `lab_results`, `invoices`, etc.).

Business logic:

- Captures operating user ID from `auth.uid()`.
- Captures `OLD` row as `old_data` JSONB on `UPDATE` and `DELETE` operations.
- Captures `NEW` row as `new_data` JSONB on `INSERT` and `UPDATE` operations.
- Inserts row into `audit_log` with transaction timestamp.

audit_ward_transfers

Purpose: Specialized audit trigger tracking inpatient movements between hospital wards and beds, maintaining chain-of-custody and rate change history.

Parameters: Trigger context on `admissions`.

Returns: `trigger`

Called from: Database trigger `trg_audit_ward_transfers` on `admissions` (`UPDATE OF ward_id, bed_id`).

Business logic:

- Detects changes where `OLD.ward_id IS DISTINCT FROM NEW.ward_id` or `OLD.bed_id IS DISTINCT FROM NEW.bed_id`.
 - Frees the previous bed (`status = 'available'`) and locks the new bed (`status = 'occupied'`).
 - Logs a structured `WARD_TRANSFER` entry in `audit_log` capturing originating ward, destination ward, previous bed, and new bed.
-

build_fhir_claim

Purpose: Compiles a complete, standardized HL7 FHIR Release 4 `Claim` resource JSON payload for a given SHA claim identifier.

Parameters:

- `p_claim_id (uuid)` — The unique SHA claim identifier in `sha_claims`.

Returns: `jsonb` (The compiled FHIR R4 Claim resource).

Called from: Edge Function `claims-dispatcher` and database trigger `auto_build_fhir_claim` (`supabase/migrations/20260813000006_sprint8_fhir_claim_resource.sql`).

Business logic:

- Retrieves claim details, patient demographics, facility KMHFL and SHA provider numbers, encounter metadata, and itemized claim lines.
 - Builds FHIR JSON structure containing:
 - `resourceType: "Claim", status: "active", use: "claim"`.
 - `patient`: Reference to FHIR `Patient`.
 - `provider`: Organization reference with KMHFL identifier.
 - `diagnosis`: Array of ICD-11 conditions mapped from `encounter_diagnoses`.
 - `item`: Array of billable service lines with tariff codes, quantities, and unit prices.
 - `total`: Total claimed monetary amount in KSh.
-

can_access_room

Purpose: Evaluates whether a specific user is authorized to enter, view queues, and conduct clinical work within a designated hospital room.

Parameters:

- `_user (uuid)` — The user UUID (`auth.uid()`).
- `_room (uuid)` — The room UUID.

Returns: `boolean` (`true` if authorized, `false` otherwise).

Called from: RLS policies on room queues and Room View components (`src/routes/rooms.$id.tsx`).

Business logic:

- Returns `true` if `_user` possesses the `admin` role in `user_roles`.
 - Returns `true` if an explicit record exists in `user_room_access` matching `user_id = _user` and `room_id = _room`.
 - Returns `false` otherwise.
-

create_encounter_from_appointment

Purpose: Converts a pre-booked outpatient appointment into an active clinical encounter upon patient arrival.

Parameters:

- `p_appointment_id (uuid)` — The appointment record UUID.

Returns: `uuid` (The newly provisioned `encounters.id`).

Called from: Appointment Calendar Workbench (`src/routes/appointments.tsx`).

Business logic:

- Fetches appointment details (patient ID, clinic provider ID, reason for visit).
 - Inserts a new row into `encounters` with `status = 'waiting'`, `encounter_type = 'outpatient'`, and notes pre-filled from appointment reasons.
 - Updates appointment status to `checked_in` and links `appointment.encounter_id`.
 - Returns the new encounter UUID for immediate room queue routing.
-

create_external_mortuary_invoice

Purpose: Generates a dedicated commercial master invoice for a deceased body brought to the mortuary from outside the hospital (police case / home death).

Parameters:

- **p_record_id** (uuid) — The **mortuary_records** identifier.
- **p_created_by** (uuid, optional) — The receiving mortician user ID.

Returns: **uuid** (The newly provisioned **invoices.id**).

Called from: Mortuary Intake Flow (**src/routes/rooms.\$id.tsx** Mortuary Room).

Business logic:

- Generates sequential invoice number (**INV-YYYY-XXXXX**).
 - Inserts a row into **invoices** with **patient_id = NULL** (external deceased) and links mortuary reference metadata.
 - Posts initial standard intake registration and cold-storage line items.
 - Returns invoice UUID.
-

dashboard_top_diseases

Purpose: Aggregates and returns the top 10 clinical diagnoses recorded across the facility within a specified date window, stratified by age band.

Parameters:

- **p_start** (date) — Start date of analysis window.
- **p_end** (date) — End date of analysis window.

Returns: Table of { **age_band text**, **disease_count integer**, **icd11_title text** }.

Called from: Executive Dashboard (**src/routes/dashboard.tsx**).

Business logic:

- Joins **encounter_diagnoses**, **encounters**, and **patients**.
 - Filters records between **p_start** and **p_end**.
 - Groups by ICD-11 condition title and computed age band (**Under 5, 5 to 59, 60+**).
 - Orders by incidence count descending, returning top morbidities for epidemiological surveillance.
-

dashboard_admitted_opd_trend

Purpose: Computes daily comparative volumes of general outpatient visits versus inpatient admissions over a selected date range.

Parameters:

- **p_start** (date) — Start date.
- **p_end** (date) — End date.

Returns: Table of { **day text**, **opd_count integer**, **admitted_count integer** }.

Called from: Executive Dashboard (**src/routes/dashboard.tsx**).

Business logic:

- Generates continuous date series between **p_start** and **p_end**.
 - Tallies outpatient encounters (**encounter_type = 'outpatient'**) and inpatient admissions (**admissions.admitted_at**).
 - Returns daily comparative array for trend chart visualization.
-

dashboard_emergency_referrals

Purpose: Aggregates emergency attendance volumes and tracks inward versus outward hospital referrals for statutory MoH 717 reporting.

Parameters:

- **p_start** (date) — Analysis start date.
- **p_end** (date) — Analysis end date.

Returns: Table of { **emergency_count integer**, **referrals_in integer**, **referrals_out integer** }.

Called from: Executive Dashboard & Reports (**src/routes/reports.tsx**).

Business logic:

- Counts encounters where `is_emergency = true`.
- Tallies `referral_direction = 'in'` and `referral_direction = 'out'`.

get_contracted_price

Purpose: Resolves the negotiated tariff for a specific medical procedure, consultation, drug, or diagnostic test under an insurance underwriter's contract.

Parameters:

- `p_insurance_provider_id (uuid)` — Underwriter UUID.
- `p_item_id (uuid)` — Catalog service or stock item UUID.
- `p_item_type (text)` — Item classification (`service`, `lab`, `radiology`, `stock`).

Returns: `numeric` (The contracted tariff amount in KSh).

Called from: Billing engine and insurance clearance workflows.

Business logic:

- Queries `contracted_prices` for matching provider, item ID, and type.
- If a contracted price is defined and active, returns the negotiated rate.
- If no contract override exists, falls back to the item's standard baseline price in `lab_test_catalog` or `stock_items`.

get_moh_705_report

Purpose: Compiles certified statutory Ministry of Health Outpatient Morbidity Reports (MOH 705A for Under 5 Years or MOH 705B for Over 5 Years) across standard disease classifications.

Parameters:

- `p_start_date (date)` — Reporting period start date.
- `p_end_date (date)` — Reporting period end date.
- `p_form_type (text, optional)` — `'705A'` (Under 5) or `'705B'` (Over 5). Defaults to `'705A'`.

Returns: Table of:

- `row_number (integer)` — Official MoH row index (Rows 1 to 45).
- `disease_name (text)` — Official MoH disease classification title.
- `icd11_code (text)` — Mapped ICD-11 reference code.
- `male_cases (integer)` — Tally of male presentations.
- `female_cases (integer)` — Tally of female presentations.
- `total_cases (integer)` — Total aggregate morbidity count.

Called from: MOH 705 Reporting Suite (`src/routes/moh.705.tsx`, `supabase/migrations/20260722120000_moh_production_sync.sql`).

Business logic:

- Queries `moh_705_disease_mappings` for the full statutory disease catalog.
- Evaluates `encounter_diagnoses` linked to encounters between `p_start_date` and `p_end_date`.
- Filters patient cohort based on age criteria (`< 5 years` for 705A; `>= 5 years` for 705B).
- Groups and sums case counts by patient biological sex (`patients.sex`).

send_lab_results_to_requesting_room

Purpose: Automatically routes a patient back into the requesting doctor's consultation room queue upon technologist verification of laboratory test results.

Parameters:

- `p_encounter_id (uuid)` — The active encounter UUID.

Returns: `text` (Target room identifier or status confirmation).

Called from: Laboratory Workbench (`src/routes/laboratory.$id.tsx`, `supabase/migrations/20260728120000_radiology_room_routing.sql`).

Business logic:

- Queries `encounter_room_visits` for `p_encounter_id` to identify the originating consultation room prior to the laboratory referral.
- Updates `encounters.current_room_id` to the originating doctor's room.

- Resets `encounters.status = 'waiting'`, creating an automated notification badge on the doctor's screen.
-

send_radiology_results_to_requesting_room

Purpose: Automatically routes a patient back into the requesting clinician's room queue upon finalization of a diagnostic radiology imaging report.

Parameters:

- `p_encounter_id (uuid)` — The active encounter UUID.

Returns: `text` (Target room identifier).

Called from: Radiology Workbench (`src/routes/radiology.$id.tsx`).

Business logic:

- Resolves the originating consultation room from `encounter_room_visits`.
 - Updates `encounters.current_room_id` to the clinician's room and sets status to `waiting`.
-

transfer_stock_between_locations

Purpose: Executes an inter-store warehouse inventory transfer from a source warehouse to a destination departmental sub-store.

Parameters:

- `source_location_id (uuid)` — Originating store UUID (e.g., Main Store).
- `destination_location_id (uuid)` — Destination store UUID (e.g., Pharmacy or ICU Store).
- `target_item_id (uuid)` — Stock item UUID.
- `transfer_quantity (numeric)` — Physical quantity to transfer.
- `note (text, optional)` — Requisition note or batch reference.

Returns: `void`

Called from: Stock Management (`src/routes/stock.tsx`).

Business logic:

- Verifies that `transfer_quantity > 0`.
 - Inserts a negative movement into `stock_movements` for `source_location_id` (`reason = 'transfer_out'`).
 - Inserts a positive movement into `stock_movements` for `destination_location_id` (`reason = 'transfer_in'`).
 - Creates a record in `stock_transfers` documenting the transaction and initiating user.
-

record_stock_usage

Purpose: Deducts consumable supplies and single-use medical devices consumed during patient clinical care or surgical procedures.

Parameters:

- `source_location_id (uuid)` — Store location UUID.
- `target_item_id (uuid)` — Stock item UUID.
- `used_quantity (numeric)` — Quantity consumed.
- `target_encounter_id (uuid, optional)` — Associated clinical encounter.
- `usage_reason (text, optional)` — Clinical procedure or department reason.
- `note (text, optional)` — Descriptive remarks.

Returns: `void`

Called from: ICU, Dialysis, and Surgical Room Workbenches.

Business logic:

- Inserts record into `stock_usage`.
 - Deducts quantity from warehouse via `stock_movements` with `reason = 'usage'`.
-

grant_stock_location_access

Purpose: Grants specific warehouse sub-store access permissions to a designated staff user.

Parameters:

- `target_user_id (uuid)` — User UUID.
- `target_location_id (uuid)` — Stock location UUID.
- `allow_view (boolean, default true)` — Permission to view inventory balances.
- `allow_request (boolean, default false)` — Permission to submit requisitions.
- `allow_approve (boolean, default false)` — Permission to approve requisitions.
- `allow_issue (boolean, default false)` — Permission to issue stock.
- `allow_receive (boolean, default false)` — Permission to receive supplier stock.

Returns: `void`

Called from: Admin Inventory Access Suite (`src/routes/admin.permissions.tsx`).

Business logic:

- Inserts or updates user privileges in `user_stock_location_access`.

has_role

Purpose: Security helper function verifying whether an authenticated user is assigned a specific hospital role.

Parameters:

- `_user_id (uuid)` — User UUID.
- `_role (public.app_role)` — Role name (e.g., `'admin'`, `'doctor'`, `'nurse'`, `'pharmacist'`).

Returns: `boolean` (`true` if assigned, `false` otherwise).

Called from: RLS policies across all database tables.

Business logic:

- Checks for matching row in `public.user_roles` where `user_id = _user_id` and `role = _role`.

user_has_permission

Purpose: Granular security helper checking whether a user possesses an explicit permission through their assigned roles.

Parameters:

- `_user (uuid)` — User UUID.
- `_perm (text)` — Permission string (e.g., `'sign_encounter'`, `'register_patient'`, `'accounting'`).

Returns: `boolean`

Called from: RLS policies and frontend navigation guards.

Business logic:

- Returns `true` if user has `admin` role.
- Joins `user_roles` and `role_permissions` to match `user_id = _user` and `permission = _perm`.

is_approved

Purpose: Foundational security gate verifying whether a staff account has been approved by an administrator.

Parameters:

- `_user_id (uuid)` — User UUID (`auth.uid()`).

Returns: `boolean` (`true` if account approved, `false` otherwise).

Called from: Global RLS policies across all 35+ tables.

Business logic:

- Queries `profiles` for `is_approved = true` or `user_roles` for active roles. Rejects unapproved user requests.

validate_and_get_icd11

Purpose: Verifies whether an ICD-11 code is locally cached and returns its official WHO MMS title and URI.

Parameters:

- `search_code (text)` — ICD-11 alphanumeric code (e.g., `'1B10'`).

Returns: Table of { `p_code text`, `p_title text`, `p_uri text`, `p_is_cached boolean` }.

Called from: Diagnosis validation triggers (`clean_and_validate_diagnosis_insert`).

Business logic:

- Checks local cache table `icd11_codes`. Returns cached record if found; otherwise indicates uncached state for Edge Function lookup.

verify_patient_identity

Purpose: Stub RPC function for verifying patient citizen identity against the National Integrated Population Registration System (IPRS) via DHA API proxy.

Parameters:

- `p_patient_id` (`uuid`) — Patient UUID.
- `p_national_id` (`text`) — Citizen National ID or Passport Number.
- `p_id_type` (`text`) — ID classification (`national_id`, `passport`, `birth_certificate`).
- `p_verified_by` (`uuid`) — Staff user initiating verification.

Returns: `jsonb` (Verification status and citizen demographic payload).

Called from: Patient Registration Form (`src/routes/register-patient.tsx`).

Business logic:

- Currently operational in STUB mode (logs verification in `patients.identity_verified`).
- Phase 3 will execute live HTTPS query to DHA IPRS gateway.

verify_practitioner

Purpose: Stub RPC validating healthcare practitioner credentials against Kenya Health Worker Regulatory Councils (KMPDC, COC, NCK, PPB, KMLTTB).

Parameters:

- `p_profile_id` (`uuid`) — Profile UUID.
- `p_council_type` (`text`) — Regulatory council name.
- `p_registration_number` (`text`) — Council license/registration number.

Returns: `jsonb` (Practitioner accreditation status).

Called from: User Profile Management (`src/routes/admin.users.tsx`).

Business logic:

- Updates `profiles.council_registration_number` and validates formatting against national council patterns.

Part 2: Supabase Edge Functions Reference

1. send-sms

- **Location:** `supabase/functions/send-sms/index.ts`
- **HTTP Method:** `POST`
- **Purpose:** Dispatches real-time SMS messages via Africa's Talking API for OTP consent codes, lab test completion notifications, and billing receipts.
- **Request Headers:**
 - **Authorization:** `Bearer <SUPABASE_ANON_OR_SERVICE_KEY>`
 - **Content-Type:** `application/json`
- **Request Payload:**

```
{
  "to": "0712345678",
  "message": "Dear Patient, your AegisCare OTP verification code is 492810. Valid for 10 minutes."
}
```

- **Business Logic:**

- Formats Kenyan telephone numbers starting with **0** or **01** to international format (+254...).
- Reads secrets **AT_USERNAME** and **AT_API_KEY**.
- Routes to Africa's Talking sandbox ([https://api.sandbox.africastalking.com/...](https://api.sandbox.africastalking.com/)) if username is **sandbox**; otherwise routes to live production gateway ([https://api.africastalking.com/...](https://api.africastalking.com/)).
- **Response Format:**

```
{
  "SMSMessageData": {
    "Message": "Sent to 1/1 Total Cost: KES 0.8000",
    "Recipients": [
      { "number": "+254712345678", "status": "Success", "cost": "KES 0.8000", "messageId": "ATXid..." }
    ]
  }
}
```

2. icd11-search

- **Location:** [supabase/functions/icd11-search/index.ts](#)
- **HTTP Method:** **POST**
- **Purpose:** Proxies live diagnostic search requests to the World Health Organization (WHO) ICD-11 API, securing API secrets server-side and caching OAuth2 bearer tokens.
- **Request Payload:**

```
{
  "query": "Malaria"
}
```

- **Business Logic:**
- Enforces minimum query length of 2 characters.
- Maintains an in-memory OAuth2 token cache with automated renewal before expiry against <https://icdaccessmanagement.who.int/connect/token>.
- Queries WHO ICD-11 MMS 2024 linearization endpoint (<https://id.who.int/icd/release/11/2024-01/mms/search>) with **useFlexisearch=true**.
- Sanitizes and strips HTML markup from WHO entity titles.
- **Response Format:**

```
{
  "results": [
    {
      "code": "1F40",
      "title": "Malaria due to Plasmodium falciparum",
      "uri": "http://id.who.int/icd/release/11/2024-01/mms/1F40"
    }
  ]
}
```

3. claims-dispatcher

- **Location:** [supabase/functions/claims-dispatcher/index.ts](#)
- **HTTP Method:** **POST**
- **Purpose:** Central claim and health information exchange router. Evaluates concluded encounters, verifies patient HIE consent, compiles FHIR bundles, and routes payloads to **dha_outbound_queue**.
- **Request Payload:**

```
{
  "encounter_id": "8fa85b1e-7b12-4c28-98e1-0d32b5f7e4a1",
  "patient_id": "4b6c7d8e-9f01-2a3b-4c5d-6e7f8a9b0c1d",
  "insurer_type": "sha_shif",
  "trigger": "encounter_closed"
}
```

- **Business Logic:**

- Authenticates user JWT.
- Verifies patient `hie_data_sharing_consented` in `patient_consent`s. If unconsented, skips HIE synchronization.
- If `insurer_type` = `'sha_shif'`: Resolves draft claim, builds FHIR claim bundle via `build_fhir_claim`, and queues in `dha_outbound_queue`.
- If `insurer_type` = `'private'`: Queues under `private_claim` handler.
- If `insurer_type` = `'cash'`: Flags internal cash ledger completion.
- **Response Format:**

```
{
  "success": true,
  "encounter_id": "8fa85b1e-7b12-4c28-98e1-0d32b5f7e4a1",
  "results": [
    { "queue_type": "fhir_sync", "handler": "FhirSyncHandler", "status": "queued", "queue_id": "..." },
    { "queue_type": "sha_claim", "handler": "ShaClaimsHandler", "status": "queued", "queue_id": "..." }
  ],
  "note": "STUB MODE – Payloads queued locally awaiting Phase 3 DHA credentials."
}
```

4. fhir-patient

- **Location:** `supabase/functions/fhir-patient/index.ts`
- **HTTP Method:** `POST`
- **Purpose:** Generates a standardized HL7 FHIR Release 4 `Patient` resource JSON payload from database demographic tables.
- **Request Payload:**

```
{
  "patient_id": "4b6c7d8e-9f01-2a3b-4c5d-6e7f8a9b0c1d"
}
```

- **Response Format:**

```
{
  "resourceType": "Patient",
  "id": "4b6c7d8e-9f01-2a3b-4c5d-6e7f8a9b0c1d",
  "meta": { "profile": ["http://hl7.org/fhir/StructureDefinition/Patient"] },
  "identifier": [
    {
      "use": "official",
      "system": "https://hisenya.org/facility/12345/patients",
      "value": "P000142"
    }
  ],
  "name": [{ "use": "official", "text": "John Kamau Mwangi", "family": "Mwangi", "given": ["John", "Kamau"] }],
  "gender": "male",
  "birthDate": "1988-04-12",
  "telecom": [{ "system": "phone", "value": "+254712345678", "use": "mobile" }],
  "address": [{ "use": "home", "city": "Nairobi", "district": "Nairobi County", "country": "KE" }],
  "managingOrganization": { "display": "AegisCare Facility" }
}
```

5. fhir-encounter

- **Location:** `supabase/functions/fhir-encounter/index.ts`
- **HTTP Method:** `POST`
- **Purpose:** Generates a standardized HL7 FHIR Release 4 `Encounter` resource detailing service class, arrival status, linked conditions, and service organization.
- **Request Payload:**

```
{
  "encounter_id": "8fa85b1e-7b12-4c28-98e1-0d32b5f7e4a1"
}
```

- **Response Format:**

```
{
  "resourceType": "Encounter",
  "id": "8fa85b1e-7b12-4c28-98e1-0d32b5f7e4a1",
  "meta": { "profile": ["http://hl7.org/fhir/StructureDefinition/Encounter"] },
  "status": "finished",
  "class": {
    "system": "http://terminology.hl7.org/CodeSystem/v3-ActCode",
    "code": "AMB",
    "display": "ambulatory"
  },
  "subject": { "reference": "Patient/4b6c7d8e-9f01-2a3b-4c5d-6e7f8a9b0c1d" },
  "diagnosis": [
    {
      "condition": { "reference": "Condition/9c8b7a6e-5d4c-3b2a-1f0e-9d8c7b6a5e4d" },
      "rank": 1
    }
  ],
  "serviceProvider": {
    "identifier": { "system": "https://hiskanya.org/facility", "value": "12345" },
    "display": "AegisCare Facility"
  }
}
```

6. fhir-condition

- **Location:** `supabase/functions/fhir-condition/index.ts`
- **HTTP Method:** `POST`
- **Purpose:** Returns an array of HL7 FHIR Release 4 **Condition** resources mapped to WHO ICD-11 concept URIs for a specified encounter.
- **Request Payload:**

```
{
  "encounter_id": "8fa85b1e-7b12-4c28-98e1-0d32b5f7e4a1"
}
```

- **Response Format:**

```
[
  {
    "resourceType": "Condition",
    "id": "9c8b7a6e-5d4c-3b2a-1f0e-9d8c7b6a5e4d",
    "meta": { "profile": ["http://hl7.org/fhir/StructureDefinition/Condition"] },
    "verificationStatus": {
      "coding": [{ "system": "http://terminology.hl7.org/CodeSystem/condition-ver-status", "code": "confirmed" }]
    },
    "code": {
      "coding": [
        {
          "system": "http://id.who.int/icd/release/11/mms",
          "code": "1F40",
          "display": "Malaria due to Plasmodium falciparum"
        }
      ]
    },
    "text": "Malaria due to Plasmodium falciparum"
  },
  "subject": { "reference": "Patient/4b6c7d8e-9f01-2a3b-4c5d-6e7f8a9b0c1d" },
  "encounter": { "reference": "Encounter/8fa85b1e-7b12-4c28-98e1-0d32b5f7e4a1" }
}
```

7. fhir-bundle

- **Location:** `supabase/functions/fhir-bundle/index.ts`
- **HTTP Method:** `POST`
- **Purpose:** Assembles a comprehensive HL7 FHIR Release 4 **Bundle** (type = **collection**) consolidating Patient, Encounter, EpisodeOfCare, Condition[], and MedicationDispense[] resources for submission to the DHA Shared Health Record (SHR).
- **Request Payload:**

```
{
  "encounter_id": "8fa85b1e-7b12-4c28-98e1-0d32b5f7e4a1"
}
```

• **Response Format:**

```
{
  "resourceType": "Bundle",
  "id": "b1c2d3e4-f5a6-7b8c-9d0e-1f2a3b4c5d6e",
  "type": "collection",
  "timestamp": "2026-08-22T12:00:00Z",
  "entry": [
    { "fullUrl": "urn:uuid:patient-id", "resource": { "resourceType": "Patient", "id": "..." } },
    { "fullUrl": "urn:uuid:encounter-id", "resource": { "resourceType": "Encounter", "id": "..." } },
    { "fullUrl": "urn:uuid:condition-id", "resource": { "resourceType": "Condition", "id": "..." } },
    { "fullUrl": "urn:uuid:dispense-id", "resource": { "resourceType": "MedicationDispense", "id": "..." } }
  ]
}
```